École de technologie supérieure
Département de génie logiciel et des TI

LOG635 - Systèmes intelligents et Algorithmes

Laboratoire 2 : Apprendre à reconnaître des objets

Étudiants	Julien Hémond HEMJ84330301 Roberto Nightingale Castillo NIGR73300101 Norman Tao TAON91090005
Cours	LOG635 - Systèmes intelligents et Algorithmes
Enseignant	José, Dolz
Session	Hiver 2024
No Groupe	01
Date	19/03/2026

Table des matières

Introduction 3
Traitement des images et extraction des primitives 3
Implémentation de réseau de neurones 5
Présentation des résultats 6
Précision, rappel et F-mesure 6
Matrices de confusion 7
Entraînement sur 80% des données 7
Entraînement sur 60% des données 8
Entraînement sur 40% des données 9
Entraînement sur 20% des données 10
Temps nécessaire pour l'entraînement et pour les prédictions 11
Interprétation des résultats 13
Conclusion 13
Bibliographie 13

Introduction

Par définition, « Un modèle IA désigne un programme entraîné sur un ensemble de données pour identifier certains schémas ou prendre des décisions sans intervention humaine » (IBM, 2023). Les données sur lesquelles il est entraîné peuvent être de plusieurs natures différentes, telles que des nombres, des caractéristiques qualitatives ou quantitatives, du texte, des sons ou bien des images.

L'objectif de ce laboratoire consiste à entraîner des modèles d'intelligence artificielle pour reconnaître des images et les catégoriser selon des classes prédéfinies. Chaque image dans l'ensemble de données d'entraînement contient un marqueur ArUco. Une étiquette identifie la classe de marqueur à laquelle chaque image appartient.

Pour réussir à entraîner des modèles d'intelligence artificielle avec ces données, il faut accomplir plusieurs étapes. Tout d'abord, il faut traiter les images, nettoyer l'ensemble de données, trouver la zone d'intérêts de chaque image et adapter les images pour qu'elles soient faciles à lire par le modèle. Ensuite, il faut implémenter un réseau de neurones efficace pour traiter ces images. Pour continuer, il est nécessaire d'entraîner ce dernier avec les données pour pouvoir présenter les résultats, tout en les comparant à ceux de différents modèles d'IA de la librairie *Scikit-learn*. Une interprétation de ces résultats permet finalement d'évaluer la performance du modèle.

Traitement des images et extraction des primitives

Premièrement, entraîner des modèles d'IA avec des images requiert de les nettoyer et de les préparer pour que le modèle puisse facilement les traiter. L'ensemble d'images fourni pour accomplir ce laboratoire est l'EnsembleA. Il contient 16 classes de marqueurs ArUco définies par une de quatre formes (cercle, hexagone, diamant ou triangle) et par la quantité de cette forme sur le marqueur. Chaque forme peut être présente deux, trois, quatre ou cinq fois dessus.

Les images de l'EnsembleA sont utilisées pour construire le jeu de données appelé EnsembleB. Cet ensemble constitue les images utilisées pour l'entraînement, la validation et le test des différents modèles du laboratoire. La transformation et le nettoyage de cet ensemble comportent plusieurs parties.

Pour commencer, en regardant l'ensemble de données, il devient évident que plusieurs images sont presque identiques, car elles ont été prises en rafale au même moment. Pour que l'entraînement se fasse sur des données différentes, on supprime manuellement certaines de ces photos qui ressemblent trop à d'autres. Au lieu, il faudra augmenter les images pour générer plus de données d'entraînement qui sont différentes. Cela permettra au modèle de généraliser au lieu de trop se spécialiser sur un angle d'image présent trop souvent.

Pour continuer, avec la librairie *OpenCV*, il faut transformer les images pour que le modèle puisse mieux les interpréter. Voici la liste des altérations faites sur chaque image avec *OpenCV*.

1. **Filtre en nuances de gris :** pour la rendre moins complexe.
2. **Filtre bilatéral :** pour réduire le bruit dans l'image tout en gardant les bordures claires.

Il est à noter que des augmentations sont effectuées plus tard après que l'ensemble de données ait été nettoyé.

Ensuite, en utilisant la même librairie, il faut isoler la zone d'intérêt (ROI) contenant le marqueur sur chaque image. Pour ce faire, un filtre *canny* est appliqué, ce qui donne une valeur de 255 aux contours détectés sur l'image et une valeur de 0 au reste. La fonction *findContours* de *OpenCV* retourne ensuite tous les contours présents sur l'image *canny*. Pour chacun des contours trouvés, le script évalue s'il contient les bonnes formes dans la quantité attendue et donne priorité aux zones d'intérêt qui ont une aire plus élevée. La ROI la plus appropriée selon ces critères est mise sur 40 par 40 pixels et enregistrée dans l'ensemble d'entraînement. Si aucune ROI potentielle n'est trouvée, cette image n'est pas ajoutée à l'ensemble de données d'entraînement. Pour environ 5% des images, la ROI sélectionnée par cet algorithme n'est pas le bon ou n'est pas de bonne qualité, donc ces images ont été supprimées ou rognées manuellement.

Par la suite, seulement 8 classes sont conservées afin que le modèle puisse faire l'entraînement plus rapidement, puisqu'il y a deux fois moins de neurones dans la couche de sortie. Afin de s'assurer que le modèle soit quand même entraîné pour généraliser le mieux possible, les 8 classes utilisées doivent être sélectionnées stratégiquement. Pour que le modèle apprenne à distinguer les formes les unes des autres, il faut garder plusieurs classes ayant le même nombre de formes, car sinon il n'aurait qu'à déterminer combien de formes sont sur l'étiquette pour la classer. Ensuite, il faut prendre les étiquettes ayant une grande quantité de formes afin que le modèle s'entraîne à distinguer les formes même sur une étiquette très dense. L'idéal aurait été de prendre les étiquettes à 4 et 5 symboles pour chaque forme, mais, après avoir nettoyé les images et éliminé les doublons, la classe Triangle5 avait beaucoup moins de données que les autres. Ce genre de problème est la raison pourquoi l'équipe fait le choix des classes après avoir nettoyé l'ensemble de données. Donc, les étiquettes de 3 et 4 symboles sont les meilleures possibles avec les données disponibles. Les classes sélectionnées sont Cercle3, Diamant3, Hexagone3, Triangle3, Cercle4, Diamant4, Hexagone4 et Triangle4.

Enfin, pour chaque image, des augmentations sont faites pour avoir une plus grande variété pour un total de 8 variantes: une originale, une image reflétée à l'horizontale, une à la verticale et une dans les 2 sens, une image assombrie et une éclaircie, une image avec une rotation antihoraire et une dans le sens horaire.

En bref, pour traiter les images, il faut supprimer les doublons, altèrer les images pour extraire les primitives, isoler la ROI et la redimensionner, éliminer 8 classes, puis faire des augmentations sur chaque image.

Implémentation de réseau de neurones

Le réseau de neurones implémenté pour la classification d'images est un réseau de neurones convolutif. Les couches se divisent en 2 types, tels que le réseau de neurones artificiel (ANN) et conventionnel (CNN). La séquence principale du réseau convolution commence par une couche Convolution, suivie par une couche Relu et finalement la couche Maxpooling.

1. La couche Convolution parcourt l'image et utilise des filtres 3x3 sur chaque pixel pour extraire les motifs de l'image.
2. La couche ReLu, la fonction d'activation, mets les valeurs négatives à 0. Ce processus a pour but d'introduire de la non-linéarité pour permettre au modèle d'apprendre les relations entre les données.
3. La couche MaxPooling réduit la taille de la donnée en gardant seulement les valeurs importantes des filtres dans une aire 2x2.

On répète cette séquence 2 fois, soit la première convolution avec 32 filtres, et la deuxième avec 64 filtres. Le réseau de neurones artificiel est composé de 1 seul layer Softmax. Ce ANN prend la sortie du réseau CNN et fait une prédiction sous forme de matrice de probabilité.

L'implémentation est vectorisée, mais il faut tout de même traiter les images en batch de 32 plutôt qu'avec le jeu de données au complet pour permettre des mises à jour du gradient fréquents.

Les hyperparamètres choisis sont :

1. Les nombres de filtres de convolution sont de 32 pour capturer les formes simples et 64 pour capture les formes complexes.
2. La taille des filtres est de 3, donc 3x3, ce qui permet de capturer et généraliser les détails locaux.
3. La taille du MaxPooling est de 2, donc 2x2, ce qui permet de réduire la taille de la donnée tout en gardant assez d'informations pour les couches suivantes.
4. La taille du batch de traitement est de 32 ce qui offre un bon équilibre entre la stabilité et efficacité pour n'importe quelle taille du jeu de données.
5. Le nombre d'époques est de 15 pour permettre à notre de modèle de converger surtout quand le jeu de données est petit.
6. La patience pour le nombre d'époques sans amélioration est de 3. Cela permet d'arrêter l'entrainement plus tôt si le modèle n'apprend pas et d'éviter le surapprentissage, surtout quand le nombre d'époques est élevées.
7. Le taux d'apprentissage démarre à 0.15, mais diminue à chaque tour d'entraînement (*epoch*), car les poids présents dans les neurones sont de plus en plus fiables, donc on ne veut pas que la rétropropagation les change trop drastiquement. Plusieurs essais ont révélé qu'une fonction qui converge quelque peu et qui passe de 0.15 (1re *epoch*) à près de 0.09 (15e *epoch*). Cette fonction est $ln_rate = 0.15 / (1 + 0.05 * epoch)$. Dans le

code, *epoch* commence à 0 jusqu'à 14, donc à 0 c'est bien $0.15 / (1 + 0) = 0.15$ et $0.15 / (1 + 0.05 * 14) = 0.0882$. En convergeant, les premières itérations bénéficient d'un taux d'apprentissage plus agressif, mais ce dernier diminue rapidement pour devenir plus conservateur et constant dans les dernières *epoch*.

La méthode de validation *holdout* est utilisée, avec une division de 60% entrainement, 20% validation et 20% test, pour évaluer la performance du modèle. Elle permet d'évaluer facilement la performance du modèle avec des données de validation jamais vues, ce qui permet de choisir le meilleur modèle et d'éviter le surapprentissage selon la précision obtenue. Ce meilleur modèle sera testé après l'entrainement avec les données de test jamais vues pour une évaluation finale.

Présentation des résultats

Les symboles de l'ensembleB, dans l'ordre des colonnes et rangées pour les matrices, sont: "Cercle3", "Cercle4", "Diamant3", "Diamant4", "Hexagone3", "Hexagone4", "Triangle3", "Triangle4".

Précision, rappel et F-mesure

	Précision	Rappel	F-mesure
RN	0.96	0.96	0.97
SVM	0.95	0.95	0.95
KNN	0.91	0.91	0.91
NN	0.95	0.95	0.95

Tableau 1 - Comparaison des mesures avec un entraînement sur 80% des données

	Précision	Rappel	F-mesure
RN	0.96	0.95	0.95
SVM	0.93	0.93	0.93
KNN	0.88	0.88	0.88
NN	0.94	0.94	0.94

Tableau 2 - Comparaison des mesures avec un entraînement sur 60% des données

	Précision	Rappel	F-mesure
RN	0.95	0.95	0.95
SVM	0.92	0.91	0.91
KNN	0.87	0.86	0.86
NN	0.92	0.91	0.91

Tableau 3 - Comparaison des mesures avec un entraînement sur 40% des données

	Précision	Rappel	F-mesure
RN	0.94	0.94	0.94
SVM	0.88	0.87	0.87
KNN	0.77	0.75	0.75
NN	0.89	0.89	0.91

Tableau 4 - Comparaison des mesures avec un entraînement sur 20% des données

Matrices de confusion

Entraînement sur 80% des données

$$\begin{bmatrix} 221 & 0 & 0 & 0 & 13 & 0 & 2 & 0 \\ 0 & 221 & 0 & 0 & 0 & 0 & 0 & 0 \\ 0 & 0 & 226 & 3 & 1 & 1 & 4 & 0 \\ 0 & 0 & 12 & 221 & 1 & 1 & 0 & 1 \\ 0 & 1 & 0 & 1 & 227 & 2 & 0 & 1 \\ 0 & 0 & 1 & 6 & 0 & 228 & 0 & 0 \\ 1 & 0 & 4 & 1 & 0 & 0 & 215 & 3 \\ 1 & 0 & 0 & 4 & 0 & 0 & 4 & 233 \end{bmatrix}$$

Figure 1 - Matrice de confusion pour le modèle "RN" avec entraînement sur 80% des données

$$\begin{bmatrix} 212 & 0 & 0 & 0 & 15 & 0 & 0 & 0 \\ 0 & 220 & 0 & 0 & 1 & 0 & 0 & 0 \\ 0 & 0 & 222 & 1 & 0 & 4 & 4 & 4 \\ 1 & 1 & 15 & 216 & 0 & 2 & 1 & 0 \\ 4 & 1 & 0 & 0 & 222 & 2 & 1 & 2 \\ 0 & 0 & 2 & 2 & 0 & 230 & 0 & 1 \\ 0 & 1 & 1 & 4 & 1 & 2 & 211 & 6 \\ 1 & 0 & 0 & 0 & 1 & 0 & 14 & 223 \end{bmatrix}$$

Figure 2 - Matrice de confusion pour le modèle "SVM" avec entraînement sur 80% des données

$$\begin{bmatrix} 212 & 1 & 4 & 0 & 19 & 0 & 0 & 1 \\ 0 & 216 & 0 & 0 & 3 & 1 & 0 & 1 \\ 0 & 0 & 221 & 4 & 0 & 5 & 2 & 3 \\ 1 & 2 & 25 & 199 & 4 & 3 & 1 & 2 \\ 6 & 1 & 0 & 3 & 216 & 4 & 0 & 2 \\ 0 & 0 & 14 & 1 & 2 & 216 & 2 & 0 \\ 0 & 1 & 3 & 0 & 0 & 2 & 207 & 13 \\ 0 & 0 & 2 & 2 & 3 & 2 & 27 & 203 \end{bmatrix}$$

Figure 3 - Matrice de confusion pour le modèle "KNN" avec entraînement sur 80% des données

$$\begin{bmatrix} 239 & 0 & 0 & 0 & 5 & 0 & 0 & 0 \\ 0 & 205 & 0 & 0 & 0 & 0 & 0 & 0 \\ 3 & 0 & 238 & 1 & 0 & 0 & 2 & 5 \\ 4 & 2 & 9 & 218 & 2 & 0 & 1 & 0 \\ 11 & 2 & 0 & 3 & 222 & 4 & 0 & 1 \\ 0 & 0 & 3 & 2 & 0 & 219 & 0 & 0 \\ 1 & 0 & 0 & 0 & 0 & 0 & 200 & 14 \\ 0 & 1 & 3 & 0 & 0 & 0 & 4 & 237 \end{bmatrix}$$

Figure 4 - Matrice de confusion pour le modèle "NN" avec entraînement sur 80% des données

Entraînement sur 60% des données

$$\begin{bmatrix} 164 & 0 & 0 & 0 & 11 & 0 & 0 & 0 \\ 0 & 161 & 0 & 0 & 0 & 0 & 1 & 0 \\ 0 & 0 & 165 & 2 & 0 & 2 & 2 & 2 \\ 0 & 0 & 6 & 166 & 4 & 0 & 0 & 0 \\ 1 & 0 & 0 & 0 & 171 & 2 & 0 & 2 \\ 0 & 0 & 3 & 1 & 1 & 173 & 0 & 2 \\ 0 & 7 & 0 & 0 & 2 & 0 & 151 & 7 \\ 0 & 1 & 0 & 0 & 0 & 1 & 4 & 182 \end{bmatrix}$$

Figure 5 - Matrice de confusion pour le modèle "RN" avec entraînement sur 60% des données

$$\begin{bmatrix} 158 & 0 & 0 & 0 & 16 & 0 & 0 & 0 \\ 0 & 161 & 0 & 0 & 0 & 0 & 0 & 1 \\ 0 & 0 & 163 & 1 & 0 & 3 & 1 & 5 \\ 2 & 2 & 11 & 155 & 2 & 3 & 0 & 1 \\ 1 & 1 & 2 & 0 & 169 & 1 & 0 & 2 \\ 0 & 0 & 8 & 1 & 1 & 168 & 0 & 1 \\ 0 & 2 & 1 & 0 & 4 & 0 & 150 & 70 \\ 0 & 1 & 0 & 0 & 0 & 0 & 20 & 168 \end{bmatrix}$$

Figure 6 - Matrice de confusion pour le modèle "SVM" avec entraînement sur 60% des données

$$\begin{bmatrix} 157 & 0 & 4 & 0 & 12 & 0 & 0 & 0 \\ 0 & 158 & 1 & 0 & 1 & 0 & 0 & 0 \\ 1 & 0 & 160 & 1 & 1 & 7 & 1 & 1 \\ 0 & 3 & 17 & 149 & 2 & 3 & 0 & 2 \\ 3 & 2 & 5 & 2 & 160 & 0 & 0 & 4 \\ 3 & 1 & 25 & 2 & 4 & 143 & 1 & 0 \\ 0 & 0 & 3 & 0 & 1 & 2 & 138 & 23 \\ 1 & 1 & 4 & 0 & 0 & 0 & 23 & 159 \end{bmatrix}$$

Figure 7 - Matrice de confusion pour le modèle "KNN" avec entraînement sur 60% des données

$$\begin{bmatrix} 138 & 0 & 0 & 0 & 19 & 0 & 2 & 0 \\ 1 & 158 & 0 & 0 & 0 & 0 & 1 & 0 \\ 0 & 0 & 169 & 1 & 1 & 1 & 2 & 0 \\ 0 & 0 & 4 & 164 & 6 & 2 & 5 & 0 \\ 3 & 0 & 0 & 3 & 165 & 2 & 1 & 0 \\ 0 & 0 & 0 & 1 & 0 & 194 & 0 & 0 \\ 0 & 1 & 3 & 0 & 0 & 0 & 163 & 10 \\ 0 & 0 & 2 & 0 & 0 & 0 & 8 & 166 \end{bmatrix}$$

Figure 8 - Matrice de confusion pour le modèle "NN" avec entraînement sur 60% des données

Entraînement sur 40% des données

$$\begin{bmatrix} 114 & 0 & 0 & 0 & 3 & 0 & 0 & 0 \\ 0 & 108 & 0 & 0 & 0 & 0 & 0 & 0 \\ 0 & 0 & 108 & 3 & 0 & 0 & 3 & 2 \\ 0 & 0 & 5 & 108 & 0 & 1 & 2 & 0 \\ 2 & 2 & 0 & 1 & 108 & 2 & 3 & 2 \\ 0 & 0 & 0 & 4 & 0 & 111 & 0 & 0 \\ 1 & 0 & 1 & 0 & 0 & 1 & 109 & 3 \\ 0 & 0 & 0 & 0 & 1 & 0 & 2 & 121 \end{bmatrix}$$

Figure 9 - Matrice de confusion pour le modèle "RN" avec entraînement sur 40% des données

$$\begin{bmatrix} 114 & 0 & 0 & 0 & 21 & 0 & 0 & 0 \\ 0 & 108 & 0 & 0 & 0 & 0 & 0 & 0 \\ 0 & 0 & 109 & 2 & 0 & 2 & 0 & 2 \\ 0 & 0 & 10 & 102 & 1 & 2 & 1 & 0 \\ 3 & 0 & 0 & 0 & 110 & 2 & 2 & 3 \\ 2 & 0 & 2 & 1 & 0 & 109 & 0 & 1 \\ 0 & 0 & 3 & 1 & 1 & 0 & 109 & 8 \\ 0 & 0 & 1 & 0 & 0 & 0 & 9 & 114 \end{bmatrix}$$

Figure 10 - Matrice de confusion pour le modèle "SVM" avec entraînement sur 40% des données

$$\begin{bmatrix} 95 & 1 & 3 & 0 & 15 & 2 & 0 & 1 \\ 0 & 106 & 0 & 0 & 1 & 90 & 1 & 0 \\ 0 & 0 & 105 & 0 & 1 & 6 & 0 & 3 \\ 0 & 1 & 15 & 95 & 1 & 4 & 0 & 0 \\ 5 & 1 & 5 & 1 & 103 & 1 & 2 & 2 \\ 1 & 0 & 15 & 2 & 0 & 97 & 0 & 0 \\ 0 & 0 & 3 & 0 & 1 & 2 & 92 & 17 \\ 1 & 2 & 5 & 0 & 0 & 1 & 8 & 107 \end{bmatrix}$$

Figure 11 - Matrice de confusion pour le modèle "KNN" avec entraînement sur 40% des données

$$\begin{bmatrix} 108 & 0 & 0 & 1 & 13 & 0 & 1 & 0 \\ 1 & 109 & 0 & 0 & 1 & 0 & 3 & 0 \\ 0 & 0 & 108 & 2 & 0 & 7 & 1 & 0 \\ 0 & 0 & 6 & 118 & 1 & 1 & 2 & 1 \\ 7 & 1 & 0 & 1 & 93 & 0 & 2 & 0 \\ 0 & 1 & 0 & 0 & 0 & 109 & 0 & 0 \\ 0 & 1 & 3 & 1 & 4 & 0 & 110 & 1 \\ 0 & 0 & 1 & 0 & 0 & 0 & 19 & 94 \end{bmatrix}$$

Figure 12 - Matrice de confusion pour le modèle "NN" avec entraînement sur 40% des données

Entraînement sur 20% des données

$$\begin{bmatrix} 54 & 1 & 0 & 0 & 3 & 0 & 0 & 0 \\ 0 & 52 & 0 & 0 & 0 & 0 & 0 & 0 \\ 0 & 0 & 57 & 0 & 0 & 0 & 0 & 0 \\ 0 & 0 & 3 & 53 & 0 & 2 & 0 & 0 \\ 1 & 0 & 0 & 1 & 54 & 1 & 1 & 1 \\ 0 & 0 & 2 & 0 & 0 & 59 & 0 & 0 \\ 0 & 0 & 2 & 0 & 0 & 0 & 54 & 5 \\ 0 & 0 & 1 & 0 & 0 & 0 & 5 & 55 \end{bmatrix}$$

Figure 13 - Matrice de confusion pour le modèle "RN" avec entraînement sur 20% des données

$$\begin{bmatrix} 52 & 0 & 0 & 0 & 5 & 0 & 0 & 1 \\ 0 & 52 & 0 & 0 & 0 & 0 & 0 & 0 \\ 0 & 0 & 49 & 1 & 0 & 3 & 0 & 4 \\ 0 & 0 & 5 & 45 & 2 & 5 & 0 & 1 \\ 3 & 0 & 1 & 0 & 51 & 1 & 1 & 2 \\ 0 & 0 & 3 & 0 & 0 & 58 & 0 & 0 \\ 0 & 0 & 2 & 0 & 1 & 2 & 50 & 6 \\ 0 & 0 & 0 & 0 & 0 & 2 & 10 & 48 \end{bmatrix}$$

Figure 14 - Matrice de confusion pour le modèle "SVM" avec entraînement sur 20% des données

$$\begin{bmatrix} 45 & 0 & 0 & 0 & 6 & 1 & 1 & 4 \\ 1 & 45 & 0 & 0 & 2 & 0 & 1 & 0 \\ 0 & 0 & 49 & 2 & 0 & 4 & 1 & 1 \\ 0 & 0 & 12 & 40 & 1 & 3 & 1 & 1 \\ 2 & 0 & 7 & 0 & 42 & 3 & 2 & 3 \\ 2 & 0 & 11 & 2 & 1 & 44 & 0 & 1 \\ 0 & 0 & 5 & 0 & 3 & 3 & 39 & 11 \\ 0 & 0 & 4 & 0 & 0 & 2 & 11 & 43 \end{bmatrix}$$

Figure 15 - Matrice de confusion pour le modèle "KNN" avec entraînement sur 20% des données

$$\begin{bmatrix} 61 & 0 & 0 & 0 & 13 & 0 & 0 & 1 \\ 0 & 63 & 0 & 0 & 0 & 0 & 0 & 0 \\ 1 & 0 & 46 & 0 & 0 & 3 & 0 & 1 \\ 1 & 0 & 5 & 47 & 2 & 0 & 0 & 0 \\ 2 & 0 & 0 & 0 & 44 & 1 & 1 & 0 \\ 0 & 0 & 2 & 1 & 0 & 46 & 0 & 0 \\ 0 & 2 & 1 & 1 & 4 & 0 & 54 & 2 \\ 0 & 0 & 0 & 0 & 0 & 0 & 9 & 53 \end{bmatrix}$$

Figure 16 - Matrice de confusion pour le modèle "NN" avec entraînement sur 20% des données

Temps nécessaire pour l'entraînement et pour les prédictions

Tableau 3 - Comparaison des mesures avec un entraînement sur 40% des données

	Temps d'entrainement	Temps de prédiction
RN	13.73 secondes	0.007 secondes
SVM	13.75 secondes	8.31 secondes
KNN	0.01 secondes	0.16 secondes
NN	14.14 secondes	1.06 secondes

Tableau 5 - Comparaison du temps d'entraînement et de prédiction sur 80% des données

	Temps d'entrainement	Temps de prédiction
RN	10.69 secondes	0.008 secondes
SVM	6.13 secondes	5.00 secondes
KNN	0.02 secondes	0.12 secondes
NN	10.67 secondes	0.93 secondes

Tableau 6 - Comparaison du temps d'entraînement et de prédiction sur 60% des données

	Temps d'entrainement	Temps de prédiction
RN	5.87 secondes	0.0027 secondes
SVM	1.78 secondes	2.49 secondes
KNN	0.004 secondes	0.04 secondes
NN	7.03 secondes	0.52 secondes

Tableau 7 - Comparaison du temps d'entraînement et de prédiction sur 40% des données

	Temps d'entrainement	Temps de prédiction
RN	5.41 secondes	0.003 secondes
SVM	0.55 secondes	0.49 secondes

KNN	0.002 secondes	0.01 secondes
NN	3.55 secondes	0.30 secondes

Tableau 8 - Comparaison du temps d'entraînement et de prédiction sur 20% des données

Interprétation des résultats

Notre modèle a une précision entre 89% et 95%, un rappel entre 89% à 95% et une F-mesure entre 91% et 95% ce qui meilleur que les modèles de SKlearn sauf pour le RN. Cette différence varie entre 1% et 5% dépendant de la taille du jeu. Toutefois, notre précision indique qu'il prédit bien les vraies négatifs et les vrais positifs. Notre rappel indique qu'il prédit un bon nombre de vrais positives selon le total des positifs. En général, on peut constater que notre modèle à une bonne performance avec le F-mesure qui est une mesure combinant la précision et le rappel.

Le SVM et KNN ont des temps d'entrainement et de prédiction plus bas que le RN et le NN, mais ces derniers ont des matrices plus diagonales indiquant qu'ils ont moins d'erreurs. Le temps d'entrainement de notre NN est similaire au RN, avec une différence de maximum 2 secondes, mais le temps de prédiction, quoique bas, est plus grand que celui du RN.

Les matrices de confusions montrent une tendance de similitude entre certains symboles du même type, comme Hexagone3 et Hexagone4 ou Triangle3 et Triangle4. Entre les différentes formes, Cercle3 et Hexagone3 sont souvent confondues entre elles.

On peut constater que le modèle est généralement moins performant sur un plus petit jeu de données, surtout comparé au RN. Le modèle a aussi besoin de plus d'époques avant de se stabiliser dans ces cas-là.

Le temps d'entrainement diminue proportionnellement à la taille du jeu, indiquant une bonne scalabilité en termes de temps. Les temps de prédictions restent rapides et similaires entre toutes les tailles de jeu.

Conclusion

Pour ce laboratoire, un modèle CNN a été implémenté pour reconnaitre des symboles ArUco. Ce modèle a été entrainé sur un EnsembleB de données constituées de 8 symboles. Ces symboles ont été choisi parmi 16 d'un EnsembleA : Cercle3, Cercle4, Diamant3, Diamant4, Hexagone3, Hexagone4, Triangle3 et Triangle4. Le CNN implémenté a été comparé a des modèles existants de *SKlearn* : KNN, SVM et RN. Parmi ces modèles, le CNN offre des meilleures performances que le KNN et SVM et un peu moins que le RN sur n'importe quelle taille de jeu de données. Ainsi, notre modèle est généralement performant pour reconnaitre les 8 symboles de l'EnsembleB.

Bibliographie

IBM. (2023, 13 septembre). Qu'est-ce qu'un modèle IA ? | IBM. Repéré à https://www.ibm.com/fr-fr/think/topics/ai-model